DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: Flipkart E-Commerce Platform Analysis

Skill Task: DBMS

Week: 10

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.

2. Find Products Above Average Price

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM Product
  3  WHERE PRICE > (SELECT AVG(PRICE) FROM Product);

PRODUCT_ID PRODUCT_NAME                                       PRICE
---------- ---------------------------------------------- ----------
       101 Laptop                                              55000
       102 Smartphone                                          25000
```

2.1 Products Below Average Price

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM Product
  3  WHERE PRICE < (SELECT AVG(PRICE) FROM Product);

PRODUCT_ID PRODUCT_NAME                                       PRICE
---------- ---------------------------------------------- ----------
       103 T-Shirt                                               799
       104 Mixer Grinder                                        2500
       105 Python Programming Book                               650
       106 Face Wash                                             350
```

3. Identify Customers with Maximum Purchases

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM Orders
  4  GROUP BY CUSTOMER_ID
  5  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1          55000
          2          25000
          4           2500
          3           1598
          5            650
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(TOTAL_PURCHASE) AS MAXIMUM_PURCHASE
  2  FROM (
  3      SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4      FROM Orders
  5      GROUP BY CUSTOMER_ID
  6  );

MAXIMUM_PURCHASE
----------------
           55000
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT c.ID, c.NAME,
  2         SUM(o.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM Customer c
  4  JOIN Orders o ON c.ID = o.CUSTOMER_ID
  5  GROUP BY c.ID, c.NAME
  6  HAVING SUM(o.TOTAL_AMOUNT) = (
  7      SELECT MAX(TOTAL_PURCHASE)
  8      FROM (
  9          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 10          FROM Orders
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

        ID NAME                 TOTAL_PURCHASE
---------- -------------------- --------------
         1 Renu                          55000
```

4. Complex Business Queries

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT DISTINCT c.ID, c.NAME, o.ORDER_ID,
  2         o.TOTAL_AMOUNT
  3  FROM Customer c
  4  JOIN Orders o ON c.ID = o.CUSTOMER_ID
  5  WHERE o.TOTAL_AMOUNT > (
  6      SELECT AVG(TOTAL_AMOUNT)
  7      FROM Orders
  8  );

        ID NAME                 ORDER_ID TOTAL_AMOUNT
---------- -------------------- ---------- ------------
         1 Renu                        401        55000
         2 Arjun                       402        25000
```

5. Find Products Whose Sales Are Above Average

```
SQL> SELECT p.PRODUCT_ID, p.PRODUCT_NAME,
  2         SUM(od.QUANTITY * od.UNIT_PRICE) AS TOTAL_SALES
  3  FROM Product p
  4  JOIN Order_Details od ON p.PRODUCT_ID = od.PRODUCT_ID
  5  GROUP BY p.PRODUCT_ID, p.PRODUCT_NAME
  6  HAVING SUM(od.QUANTITY * od.UNIT_PRICE) > (
  7      SELECT AVG(PRODUCT_SALES)
  8      FROM (
  9          SELECT SUM(QUANTITY * UNIT_PRICE) AS PRODUCT_SALES
 10          FROM Order_Details
 11          GROUP BY PRODUCT_ID
 12      )
 13  );

PRODUCT_ID PRODUCT_NAME                                       TOTAL_SALES
---------- -------------------------------------------------- -----------
       101 Laptop                                                   55000
       102 Smartphone                                               25000
```

6. Customer Purchase Report

```
SQL> SELECT c.ID AS CUSTOMER_ID,
  2         c.NAME AS CUSTOMER_NAME,
  3         COUNT(o.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         NVL(SUM(o.TOTAL_AMOUNT), 0) AS TOTAL_PURCHASE,
  5         NVL(AVG(o.TOTAL_AMOUNT), 0) AS AVERAGE_ORDER_VALUE
  6  FROM Customer c
  7  LEFT JOIN Orders o ON c.ID = o.CUSTOMER_ID
  8  GROUP BY c.ID, c.NAME
  9  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_PURCHASE
----------- -------------------- ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
          1 Renu                                1          55000
              55000

          2 Arjun                               1          25000
              25000

          4 Rahul                               1           2500
               2500


CUSTOMER_ID CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_PURCHASE
----------- -------------------- ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
          3 Priya                               1           1598
               1598

          5 Keerthana                           1            650
                650
```

7. Advanced Product Sales Report

```
SQL> SELECT c.ID AS CUSTOMER_ID,
  2         c.NAME AS CUSTOMER_NAME,
  3         COUNT(o.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         NVL(SUM(o.TOTAL_AMOUNT), 0) AS TOTAL_PURCHASE,
  5         NVL(AVG(o.TOTAL_AMOUNT), 0) AS AVERAGE_ORDER_VALUE
  6  FROM Customer c
  7  LEFT JOIN Orders o ON c.ID = o.CUSTOMER_ID
  8  GROUP BY c.ID, c.NAME
  9  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_PURCHASE
----------- -------------------- ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
          1 Renu                                1          55000
              55000

          2 Arjun                               1          25000
              25000

          4 Rahul                               1           2500
               2500


CUSTOMER_ID CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_PURCHASE
----------- -------------------- ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
          3 Priya                               1           1598
               1598

          5 Keerthana                           1            650
                650
```

8. Category-wise Advanced Sales Report

```
SQL> SELECT p.PRODUCT_ID,
  2         p.PRODUCT_NAME,
  3         COUNT(DISTINCT od.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(od.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  5         SUM(od.QUANTITY * od.UNIT_PRICE) AS TOTAL_SALES,
  6         AVG(od.UNIT_PRICE) AS AVERAGE_SELLING_PRICE
  7  FROM Product p
  8  JOIN Order_Details od ON p.PRODUCT_ID = od.PRODUCT_ID
  9  GROUP BY p.PRODUCT_ID, p.PRODUCT_NAME
 10  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID PRODUCT_NAME                                       NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- --------------------
       101 Laptop                                                            1
                 1       55000                55000

       102 Smartphone                                                        1
                 1       25000                25000

       104 Mixer Grinder                                                     1
                 1        2500                 2500


PRODUCT_ID PRODUCT_NAME                                       NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- --------------------
       103 T-Shirt                                                           1
                 2        1598                  799

       105 Python Programming Book                                           1
                 1         650                  650
```